Quantyze: Trees, Trades, and Books

Simulating a Limit Order Book with Tree-Based Data Structures and an ML Agent

Cade McNelly Nicolas Miranda Cantanhede Sahand Samadirand

March 31, 2026

1. Introduction

As a group of CS students interested in finance, we have spent a lot of time looking at stock charts. It does not take long to notice a major disconnect between the price shown on the screen and the mechanics operating underneath it. A market interface may display a stock at a certain level, but that does not mean an order can be filled there instantly or in unlimited size. In practice, fills depend on liquidity, queue position, and the slippage created when an order moves through the book.[1]

That gap between a quoted price and an executable price is one of the main ideas that motivated this project. It is also a very natural place to apply CSC111 ideas such as tree-based data structures, simulation, and algorithmic state updates. Quantyze studies that gap by simulating a limit order book in Python.[2] We represent bids and asks as separate binary search trees of price levels, maintain FIFO priority within each level, replay event streams through a matching engine, and compute summary statistics about fills, cancellations, spread, midpoint, and slippage. On top of the simulator, we added a small machine learning component that observes book-state features and classifies market actions in the short term.

Our project goal is to build a tree-based market simulator that can replay realistic order flow, expose the mechanics of price-time-priority matching, and train a small neural network to classify short term market actions as buy, sell, or hold from order-book state.

The project kept the same core direction as our proposal, but the final version organizes the submission around a browser display launched from `main.py`, with a terminal-menu option for the same simulator and training logic. In the language of our original proposal, we still see the project as an attempt to close the gap between what a trader sees on a chart and what actually has to happen for an order to be filled.

2. Datasets

Not all data in Quantyze plays the same role. It uses three distinct **data-source roles**, as described below:

- **synthetic scenarios** for simulator-first replay demos
- **packaged internal CSVs** for fast and reproducible classifier retraining
- **paired raw LOBSTER files** for the real-data preprocessing and feature-building path

[1] Gould et al. (2013) give a useful overview of price-time priority, queue position, and other core limit order-book mechanics: `https://arxiv.org/abs/1012.0349`

[2] Cont and de Larrard (2013) model price dynamics directly from the evolving bid and ask queues, which matches our focus very nicely state and order flow jointly shape execution outcomes: `https://arxiv.org/abs/1104.4596`

2.1 Synthetic Event Data

We generated our own synthetic event streams for testing and demonstration. These are created inside `data_loader.py` and include three scenarios:

- `balanced`
- `low_liquidity`
- `high_volatility`

These synthetic datasets use the same canonical event format as the rest of the project: `timestamp`, `order_id`, `side`, `order_type`, `price`, and `quantity`. Their role is **simulation only**. They are the fastest way to see fills, cancellations, spread changes, and book updates without waiting for a large dataset to load.

2.2 LOBSTER Sample Data

For the real-data path of the machine learning portion of the project, we used LOBSTER sample order-book data from the official LOBSTER sample page.[3] Each LOBSTER sample comes as a **pair of files**:

- a message file containing timestamped market events
- an order-book file containing depth snapshots

Our code supports raw LOBSTER message and order-book CSVs. The training flow is pointed at the **message file**, and the loader infers the paired order-book file automatically from the filename. From the message file, we use normalized information corresponding to timestamp, event type, order id, size, price, and direction. The loader also recognizes message types that should be preserved as annotations rather than replayed directly.

From the paired order-book file, our current feature pipeline uses only the top two bid levels and top two ask levels, even when a file contains deeper book data. In practice, most of the meaningful data comes from these two levels, and so that's why we chose to use that approach.

During development, we used public LOBSTER sample files such as AAPL, AMZN, and GOOG to validate the real-data pipeline. In the final submission, we ship one official Apple 5-level sample pair inside the dataset zip. This packaged LOBSTER option is **optional**. It demonstrates the real-data integration in our project, but it is much larger and slower to load than the short internal packaged dataset.

2.3 Submission Dataset Package

For the final submission, the dataset zip is `quantyze_datasets.zip`. It contains:

- `sample_internal.csv`
- `huge_internal.csv`
- `aapl_lobster_2012-06-21_message_5level_sample.csv`
- `aapl_lobster_2012-06-21_orderbook_5level_sample.csv`
- `instructions.txt`
- `model.pt`
- `training_metrics.json`

[3] `https://data.lobsterdata.com/info/DataSamples.php`

After extraction, these files are expected to sit beside `main.py`. Both the browser UI and the terminal menu look for the packaged datasets saved artifacts there.

The two packaged internal CSVs, `sample_internal.csv` and `huge_internal.csv`, are local datasets prepared by our group in the project's event format so the TA can run smooth retraining without extra preprocessing. The submission also includes `generate_internal_datasets.py`, which reproduces those two packaged internal CSVs exactly; (not required for the main run)

The intended roles of these packaged files are:

- `sample_internal.csv`: the **quick** retraining check
- `huge_internal.csv`: the **large internal** retraining demo, which takes longer to load and train
- `aapl_lobster_2012-06-21_message_5level_sample.csv`: the **large raw LOBSTER message-file** demo
- `aapl_lobster_2012-06-21_orderbook_5level_sample.csv`: the paired order-book file required by the raw LOBSTER path

The package also includes saved metrics and a baseline checkpoint, so the TA can use the model path in simulation without retraining.

Please note that the packaged datasets and baseline artifacts are local, but the `Charts` tab loads `Chart.js` from a standard CDN, so the intended TA path assumes a normal internet connection for the complete UI, with the terminal menu serving a backup option if needed.

3. Computational Overview

3.1 Core Data Representation

The central data structure in Quantyze is the limit order book. We represent the book with two binary search trees:

- one tree for bids
- one tree for asks

This logic is implemented across `book_tree.py`, `price_level.py`, and `order_book.py`. Each node in a tree is a `PriceLevel` object. A price level stores:

- a price
- aggregate resting volume at that price
- a FIFO queue of resting orders

The trees let us preserve sorted price structure while still supporting efficient best-bid and best-ask access. The FIFO queue inside each price level enforces time priority among orders resting at the same price. The queue is implemented as a doubly-linked list with a parallel `order_id`-to-node dictionary, which gives O(1) cancellation by order id in addition to O(1) enqueue and dequeue. This combination helped us model price-time-priority matching, which was the main algorithmic goal of our project.[4]

[4] Gould et al. (2013) provide a comprehensive treatment of price-time priority and order-book microstructure dynamics: `https://arxiv.org/abs/1012.0349`

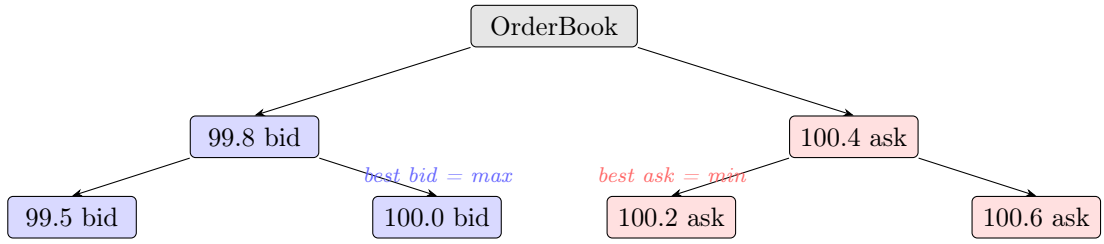

Figure 1: The `OrderBook` holds two `BookTree` instances. The bid tree's maximum-price node is the best bid; the ask tree's minimum-price node is the best ask.

3.2 Event Replay and Matching

All incoming data is converted into a common `Event` representation in `orders.py`. The `DataLoader` in `data_loader.py` either:

- loads a dataset from internal CSV or raw LOBSTER files, or
- generates a synthetic scenario

Those events are then replayed through `EventStream` in `event_stream.py`, which emits them one at a time into the `MatchingEngine` in `matching_engine.py`. The matching engine handles limit orders, market orders, and cancellations. It updates the `OrderBook`, performs partial and full fills, removes empty price levels when necessary, and records execution information used for later metrics.

At the end of a normal run, the simulation runtime reports:

- total filled quantity
- fill count
- cancel count
- average slippage
- final spread
- final midpoint
- agent overlay mark-to-market P&L if a trained checkpoint is active

The same simulation and training state is processed in two ways:

- the browser UI launched by `main.py`
- the optional terminal menu opened through `main(run_ui=False)`

The saved artifacts include:

- `model.pt`
- `training_metrics.json`
- `latest_model.pt`
- `latest_training_metrics.json`
- `latest_training_data.csv`
- `active_model.json`
- `log.json`

The packaged baseline artifacts are `model.pt` and `training_metrics.json`. The remaining files are created or updated as you simulate, retrain, change overlay mode, or inspect the latest execution log.

3.3 Data Processing and Training

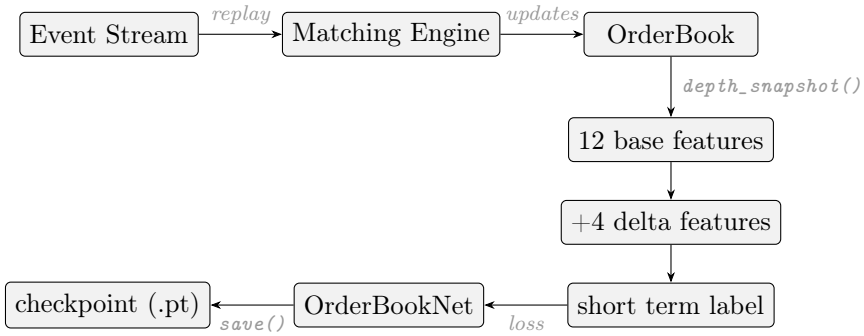

Figure 2: This is how the training data flows: events are replayed through the matching engine to keep the `OrderBook` live. At each step, a 16-D feature vector and a short term label are extracted. `OrderBookNet` is trained with class-weighted cross-entropy and the best-validation-loss state is saved as a checkpoint.

The machine learning pipeline is built mainly in `data_loader.py`, `neural_net.py`, and `training.py`.

The feature pipeline uses 16 features per example. The first 12 are instantaneous book-state features extracted by `DataLoader._feature_vector_from_levels`:

1. best bid price
2. best bid size
3. best ask price
4. best ask size
5. spread
6. mid-price
7. level-1 imbalance
8. second bid price
9. second bid size
10. second ask price
11. second ask size
12. event-side feature

The remaining 4 are one-step history-delta features appended by `DataLoader.augment_feature_vector`:

13. one-step best bid delta
14. one-step best ask delta
15. one-step mid-price delta

16. one-step imbalance delta

The label policy is **short-term based**. For each example, we compare the current midpoint to the midpoint 50 events later:

- if the future midpoint is higher by more than the threshold, the label is `buy`
- if it is lower by more than the threshold, the label is `sell`
- otherwise the label is `hold`

This idea is implemented in `DataLoader.build_training_dataset` and related helper methods. The final training run in `training.py` computes train/validation splits, standardizes features with training-set mean and standard deviation, applies class-weighted cross-entropy, trains a PyTorch network, and saves the latest training artifacts:

- `latest_model.pt`
- `latest_training_metrics.json`
- `latest_training_data.csv`

The shipped baseline saved-state artifacts remain `model.pt` and `training_metrics.json`, so the TA can always return to a known working checkpoint.

3.4 User Interactivity

The TA-facing interface is a browser UI launched by `main.py` and served by `web.py`. Running `main.py` in PyCharm starts the browser workflow at `http://127.0.0.1:9000`. The browser UI has four tabs:

- `Simulate`
- `Train`
- `Artifacts`
- `Charts`

Those tabs expose the same underlying simulator and training pipeline from different angles. `Simulate` chooses a synthetic scenario or CSV replay, controls replay speed, and selects the overlay mode. `Train` runs the packaged internal datasets, the packaged LOBSTER message-file path, or a custom CSV path. `Artifacts` reports the active overlay state, saved checkpoints, metrics files, and latest training outputs. `Charts` renders order-book depth and execution-price history after a simulation. The backup terminal path uses `terminal_menu.py`, but the desired path is the browser UI for better interactivity and visualization of the charts.

The active overlay choice is stored in `active_model.json`, which makes the browser run persistently across runs.

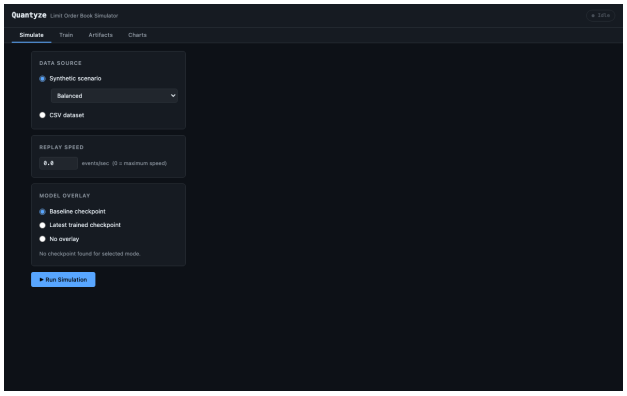

Figure 3: Simulate tab: scenario selector, overlay mode, and replay controls.

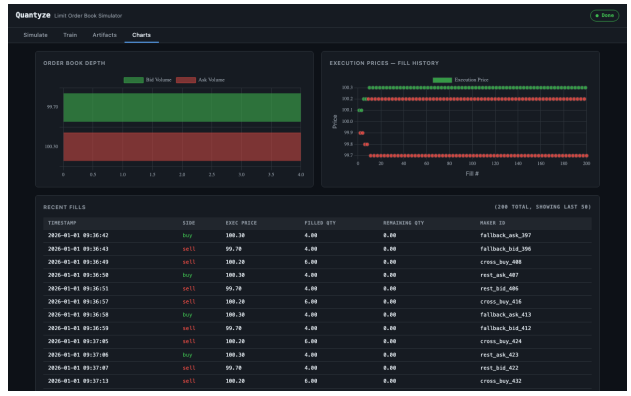

Figure 4: Charts tab: order-book depth and execution-price history after a run.

3.5 Overall Computational Complexity

Quantyze demonstrates computational complexity in all three major ways highlighted by the project rubric.

- **Non-trivial dataset processing and model training:** the system supports multiple input formats, patches LOBSTER order-book files, separates replayable and non-replayable events, builds a shared training dataset, normalizes features, computes class weights, and saves checkpoints and metric artifacts.
- **Non-trivial algorithms implemented by us:** the BST-based order book, FIFO price-level queues, price-time-priority matching engine, and cancellation/indexing logic are all custom project code rather than thin wrappers around third-party libraries.
- **Non-trivial reporting and interactivity:** the browser UI, terminal menu option, saved-state artifacts and metrics, and execution-log reporting make the program very interactive.

4. Instructions for Obtaining Datasets and Running the Program

As described before, the main workflow is the browser UI, with an additional terminal menu option. The most important distinction is that **simulation source** and **model source** are separate:

- the **simulation source** is the event stream being replayed, such as `balanced`, `low_liquidity`, `high_volatility`, or a custom CSV
- the **model source** is the classifier overlay currently selected for inference, such as the shipped `model.pt` baseline or a newly trained `latest_model.pt`

Keeping them separate allows testing the same classifier overlay under different market conditions and retrain a model without overwriting the shipped baseline artifacts. We found this distinction important because the simulator is the core system, while the model is an optional layer on top of it.

4.1 Installing Libraries

Install the libraries listed in `requirements.txt` before running the program.

4.2 Obtaining Datasets

The TA downloads `quantyze_datasets.zip` from our final MarkUs submission and extracts it beside `main.py`. After extraction, the project should see:

- `sample_internal.csv`
- `huge_internal.csv`
- `aapl_lobster_2012-06-21_message_5level_sample.csv`
- `aapl_lobster_2012-06-21_orderbook_5level_sample.csv`
- `instructions.txt`
- `model.pt`
- `training_metrics.json`

The shortest path is using `sample_internal.csv`. The larger internal dataset and the packaged LOBSTER pair are both slower options, but also working options.

As the TA trains or changes overlay state, the program may also create or update:

- `latest_model.pt`
- `latest_training_metrics.json`
- `latest_training_data.csv`
- `active_model.json`
- `log.json`

4.3 Workflow Overview

1. extract `quantyze_datasets.zip` beside `main.py`
2. open the project in PyCharm
3. run `main.py`
4. open `http://127.0.0.1:9000`
5. use the `Simulate`, `Train`, `Artifacts`, and `Charts` tabs

This path makes four things explicit at all times:

- what event source is being replayed
- which checkpoint, if any, is active
- where that checkpoint came from
- what to do next: simulate, train, or inspect artifacts

4.4 Recommended TA Demo

The clearest way to understand the project is the following walkthrough after running `main.py` and opening the printed local URL:

1. Start on the `Simulate` tab and run the default `balanced` scenario. This is the fastest simulator-first path and produces the same summary metrics described later in this section.

2. On the same tab, you can run `low_liquidity` and `high_volatility`. These scenarios show that the same overlay can be evaluated under different event streams without changing the core matching engine.

3. Open `Train`. For a short demonstration, use `sample_internal.csv`. For a larger demonstration, use `huge_internal.csv`.
   For the real-data path, use `aapl_lobster_2012-06-21_message_5level_sample.csv`. The latter two are both much larger and take longer to load. After training, the UI writes the latest output files and exposes the new checkpoint in the saved artifact status.

4. Open `Artifacts`. Check the overlay status, saved files, and baseline versus latest metrics.

5. Open `Charts` after a simulation. This shows order-book depth and execution-price history based on the replay that just ran. It changes for different datasets, events streams, etc. So if you want check out the different charts!

6. Re-run `balanced` under a different overlay to isolate the difference between market replay and agent overlay behaviour.

7. Optional data generation check: run `generate_internal_datasets.py` to regenerate `sample_internal.csv` and `huge_internal.csv`.

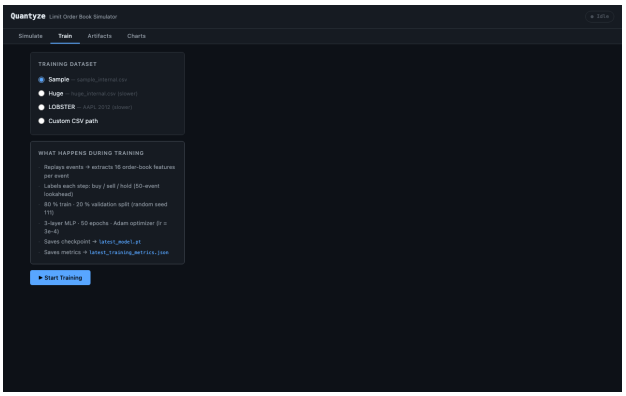

Figure 5: Train tab: dataset selector, training progress, and live accuracy readout.

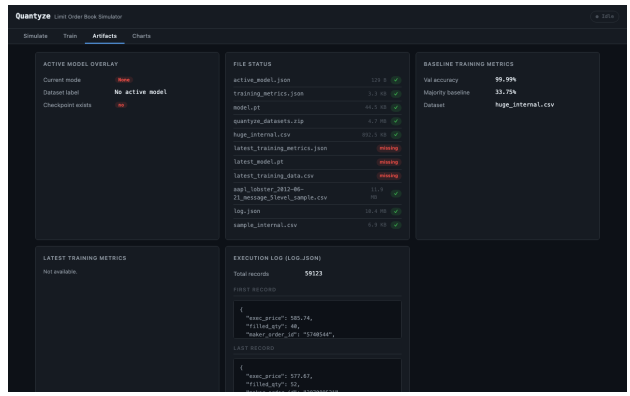

Figure 6: Artifacts tab: overlay status, saved checkpoint paths, and baseline vs. latest metrics.

4.5 How the TA Should Interpret the Output

The browser UI summaries and terminal menu contain the same two classes of information, and keeping them separate is the key to understanding Quantyze.

Simulation Run Configuration

Before each run, the program prints:

- the **event source**, such as `synthetic balanced` or a custom replay path
- the **simulation overlay mode**, such as `baseline`, `latest`, or `none`
- the **simulation overlay path**
- the **overlay used**, such as the packaged baseline checkpoint or a newly trained dataset like `sample_internal.csv`

This tells which market is being replayed and which classifier overlay, if any, is observing it. They are related, but they are not the same thing!

Matching-Engine Metrics

The following summary values describe the replay itself:

- `Total Filled`: total executed quantity produced by the matching engine
- `Fill Count`: number of executions
- `Cancel Count`: number of cancellations processed
- `Average Slippage`: average execution-price deviation under the current slippage calculation
- `Spread`: final bid-ask spread
- `Mid Price`: final midpoint

These values are properties of the event stream and the matching engine. If the TA replays the same scenario with `baseline`, `latest`, and `none`, the market metrics should stay the same because the underlying event stream is unchanged.

Agent Overlay Metric

If an active checkpoint is loaded, the program also prints `Agent Overlay Mark-to-Market P&L`. This value needs to be interpreted carefully.

It is **not** a claim that the model is a profitable trading strategy. In the current system, the neural network is trained from the midpoint move 50 events ahead and maps each step to `buy`, `sell`, or `hold`. During simulation, that prediction is converted into a simple paper-trading overlay that increments or decrements inventory by one unit whenever the simulator observes fills. As a result:

- a strong classification accuracy does not automatically imply a strong trading P&L
- a negative mark-to-market value can reflect inventory accumulation in a thin market rather than a broken simulator
- the ML component should be interpreted as a proof-of-concept prediction overlay, not as a finished trading strategy

For example, `low_liquidity` often ends with a wider spread and fewer fills than `balanced`. A model can therefore finish with non-zero inventory and a negative mark-to-market value even when the matching engine itself is behaving correctly.

Training Metrics

The `Train` tab and the terminal menu report validation accuracy, majority-class baseline, per-class recall, and the confusion matrix. These values measure how well the classifier reproduces the label rule derived from future midpoint movement. They do not directly measure whether the simulated agent will earn positive P&L on every synthetic scenario.

4.6 Optional Raw LOBSTER Check

If the TA wants to test the raw LOBSTER path, they do **not** need to download external files. They can use the packaged training-menu option for `aapl_lobster_2012-06-21_message_5level_sample.csv`. The paired order-book path is inferred automatically from that message filename, which maps to `aapl_lobster_2012-06-21_orderbook_5level_sample.csv`.

The same loader logic also works for a custom raw LOBSTER message CSV path: it infers the paired order-book path by replacing `_message_` with `_orderbook_`.

5. Changes from Proposal to Final Submission

The final project is cleaner and narrower than the proposal. The simulator remained the core implementation, and the ML component stayed as an overlay rather than becoming the center of the project.

On the implementation side, we ended with a clearer module split than we originally imagined: `config.py` holds shared constants and overlay state, `simulation.py` owns replay orchestration, `training.py` owns classifier training, `api_payloads.py` owns browser-API response serialization, `web.py` serves the browser UI, `terminal_menu.py` owns the backup terminal flow, and `main.py` is now a small entry point.

Our frontend plan also changed. We decided to move away from the React/Plotly-style interface, and use a Flask/browser UI instead. That choice let us focus on the tree-based order book, matching engine, and training pipeline while still giving the TA a clear PyCharm-friendly workflow.

The data plan became more concrete as well. Instead of treating external data abstractly, the final submission cleanly separates synthetic replay scenarios, packaged internal CSVs for fast retraining, and one packaged raw LOBSTER pair for the real-data preprocessing path.

6. Discussion

Our goal was to model the mechanics of a limit order book using trees and to connect that simulator to a simple market-action classifier. The final project does meet that goal.

On the simulation side, the output is meaningful and easy to interpret. In the current default `balanced` run, the simulator reports 1748.00 total filled volume, 374 fills, 128 cancellations, a final spread of 0.60, and a final midpoint of 100.00. Those values show that the event sequence is properly generating executions, consuming liquidity, and rebuilding the book over time. That mattered to us from the beginning, because one of the main motivations for the project was to show the difference between a quoted market price and the mechanics to actually process the order.

We also think the project succeeds as a data-structures project. The binary search trees are the central representation of the market. They are intrinsic to maintain sorted price levels, extract the best bid and best ask, remove empty levels, and preserve FIFO priority within a level all fit the tree-based design naturally. Furthermore, their specific structure makes the program run more efficiently, as we learned in class!

The machine learning component works, but it needs to be interpreted carefully. The packaged baseline checkpoint and its accompanying metrics artifact now show a validation accuracy of approximately 0.81644 compared to a majority-class baseline of approximately 0.66184 on `huge_internal.csv`. These values are useful for confirming that the training model works correctly, but they are still tied to a packaged internal dataset rather than complex real-market circumnstances.

When we tested on public LOBSTER sample data during development, the task was much harder and performance was only modestly above a naive baseline. For that reason, we do not present the current model as a strong predictive system for real markets. In this project, its role is better understood as a small proof of concept that we are able to connect limit order books to a machine learning model.

That still answers the project goal in a meaningful way. The goal was never just to maximize predictive accuracy. It was to build a full computational pipeline from market events to order-book state to derived features to model output. In that sense, we consider the ML side to be successful.

We also ran into clear limitations. Synthetic data is easy to test and but is not enough to justify a strong predictive claim. Real LOBSTER data is much richer, but it brings preprocessing issues, class-balance issues, and a harder learning problem. The model itself is a small Pytorch feed-foward neural network, which we found reasonable for the scope of this project and the course., but it limits what the model can learn about market dynamics. Finally, our presentation scope narrowed over the course of the project. We planned to integrate a more polished UI layer, but most of our time went into the simulation engine, dataset handling, and the training side

If we were to continue the project, the next steps would be to improve the visualization layer, add a stronger sequence-aware model, and evaluate the trained agent with more realistic trading metrics across multiple market days. Even with those limitations, we think the program is very successful as it combines a tree-based architecture with simulation, event processing and model training. More importantly, it still reflects the original motivation behind the project: to show the hidden mechanics of a financial market.

We hope you enjoyed the project as much as we did!

7. References

1. LOBSTER. (n.d.). *Sample data*. Retrieved March 31, 2026, from `https://data.lobsterdata.com/info/DataSamples.php`
2. PyTorch. (n.d.). *PyTorch*. Retrieved March 31, 2026, from `https://pytorch.org/`
3. NumPy Developers. (n.d.). *NumPy*. Retrieved April 2, 2026, from `https://numpy.org/`
4. Flask. (n.d.). *Flask*. Retrieved April 2, 2026, from `https://flask.palletsprojects.com/`
5. Investopedia. (n.d.). *Limit order book*. Retrieved March 31, 2026, from `https://www.investopedia.com/terms/l/limitorderbook.asp`
6. Investopedia. (n.d.). *Depth of market (DOM)*. Retrieved March 31, 2026, from `https://www.investopedia.com/terms/d/depth-of-market.asp`
7. QuestDB. (n.d.). *Limit order book*. Retrieved March 31, 2026, from `https://questdb.com/glossary/limit-order-book/`
8. Gould, M. D., Porter, M. A., Williams, S., McDonald, M., Fenn, D. J., & Howison, S. D. (2013). Limit order books. *Quantitative Finance*, 13(11), 1709–1742. Retrieved March 31, 2026, from `https://arxiv.org/abs/1012.0349`
9. Cont, R., & de Larrard, A. (2013). Price dynamics in a Markovian limit order market. *SIAM Journal on Financial Mathematics*, 4(1), 1–25. Retrieved April 2, 2026, from `https://arxiv.org/abs/1104.4596`